

Chapter 5: Web and Database Programming with .Net

Active Server Page (ASP): Active Server Pages (ASP) is a server-side scripting technology that can be used to create dynamic and interactive Web applications. An ASP page is an HTML page that contains server-side scripts that are processed by the Web server before being sent to the user's browser. We can combine ASP with Extensible Markup Language (XML), Component Object Model (COM), and Hypertext Markup Language (HTML) to create powerful interactive Web sites. Server-side scripts run when a browser requests an .asp file from the Web server. ASP is called by the Web server, which processes the requested file from top to bottom and executes any script commands. It then formats a standard Web page and sends it to the browser.

An Example of classical asp page. This example displays the words "Hello World".

```
<%@ Language=VBScript %>
<html> <head> <title>Example 1</title> </head>
    <body>
        <% FirstVar = "Hello world!" %>
        <%=FirstVar%>
    </body>
</html>
```

ASP.Net

When the first version of the .NET Framework was released nearly a decade ago, it was the start of a radical new direction in software design. Inspired by the best of Java, COM, and the Web, and informed by the mistakes and limitations of previous technologies, Microsoft set out to “hit the reset button” on their development platform. The result was a set of surprisingly mature technologies that developers could use to do everything from building a Windows application to executing a database query, and a web-site-building tool known as ASP.NET.

Today, ASP.NET is as popular as ever, but it's no longer quite as revolutionary. And, although the basic functionality that sits at the heart of ASP.NET is virtually the same as it was ten years ago, Microsoft has added layers of new features and higher-level coding abstractions. It has also introduced at least one new direction that competes with traditional ASP.NET programming, which is called ASP.NET MVC.

Seven key Pillars of ASP.Net

1. ASP.NET Is Integrated with the .NET Framework: The .NET Framework is divided into an almost painstaking collection of functional parts, with tens of thousands of types (the .NET term for classes, structures, interfaces, and other core programming ingredients). The massive collection of functionality that the .NET Framework provides is organized in a way those traditional Windows programmers will see as a happy improvement. Each one of the thousands of classes in the .NET Framework is grouped into a logical, hierarchical container called a namespace.

Different namespaces provide different features. Taken together, the .NET namespaces offer functionality for nearly every aspect of distributed development from message queuing to security. This massive toolkit is called the class library.

2. ASP.NET Is Compiled, Not Interpreted: ASP.NET applications, like all .NET applications, are always compiled. In fact, it's impossible to execute C# or Visual Basic code without it being compiled first. .NET applications actually go through two stages of compilation. In the **first stage**,

the C# code is compiled into an intermediate language called Microsoft Intermediate Language (MSIL), or just IL. This first step is the fundamental reason that .NET can be language-interdependent. Essentially, all .NET languages (including C#, Visual Basic, and many more) are compiled into virtually identical IL code. This first compilation step may happen automatically when the page is first requested. The compiled file with IL code is an assembly.

The second level of compilation happens just before the page is actually executed. At this point, the IL code is compiled into low-level native machine code. This stage is known as **just in time (JIT)** compilation, and it takes place in the same way for all .NET applications (including Windows Applications, for example). Figure below shows this two-step compilation process.

.NET compilation is decoupled into two steps in order to offer developers the most convenience and the best portability. Before a compiler can create low-level machine code, it needs to know what type of operating system and hardware platform the application will run on (for example, 32-bit or 64-bit Windows). By having two compile stages, we can create a compiled assembly with .NET code and still distribute this to more than one platform.

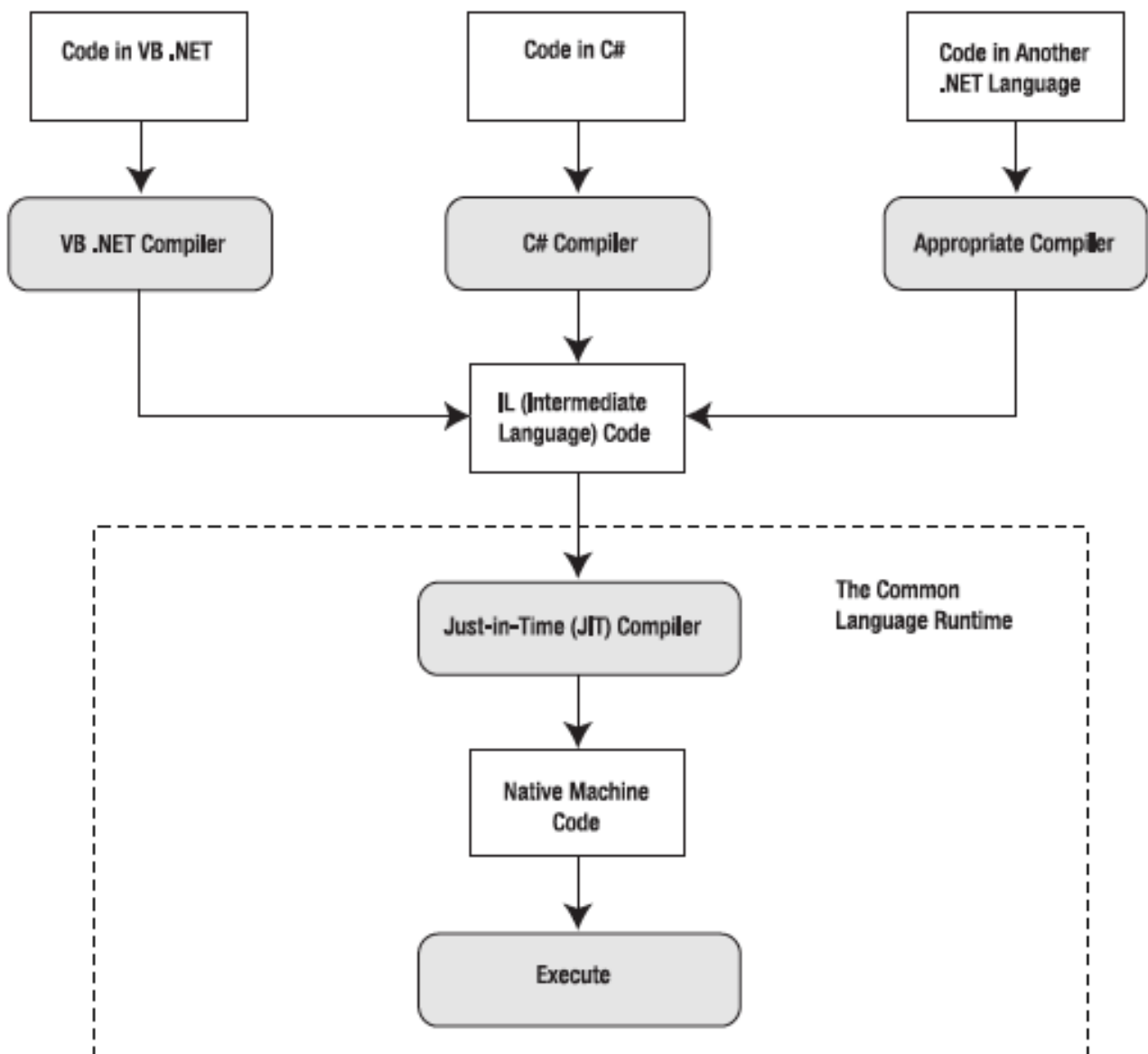


Fig: Compilation of asp.net web page

JIT compilation probably wouldn't be that useful if it needed to be performed every time a user requested a web page from our site. Fortunately, ASP.NET applications don't need to be compiled every time a web page is requested. Instead, the IL code is created once and regenerated only when

the source is modified. Similarly, the native machine code files are cached in a system directory that has a path.

3. ASP.NET Is Multilanguage: Many programmers 'll probably opt to use one language over another when one develop an application, that choice won't determine what can accomplish with web applications. That's because no matter what language we use, the code is compiled into IL.

4. ASP.NET Is Hosted by the Common Language Runtime: Perhaps the most important aspect of the ASP.NET engine is that it runs inside the runtime environment of the CLR. The whole of the .NET Framework—that is, all namespaces, applications, and classes—is referred to as managed code. Some benefits of CLR are Automatic memory management and garbage collection, type safety, Extensible metadata, structured error handling, Multithreading etc.

5. ASP.NET Is Object-Oriented: ASP provides a relatively feeble object model. It provides a small set of objects; these objects are really just a thin layer over the raw details of HTTP and HTML. On the other hand, ASP.NET is truly object oriented.

Not only does our code have full access to all objects in the .NET Framework, but we can also exploit all the conventions of an OOP (object-oriented programming) environment. For example, we can create reusable classes, standardize code with interfaces, extend existing classes with inheritance, and bundle useful functionality in a distributable, compiled component.

One of the best examples of object-oriented thinking in ASP.NET is found in server-based controls. Server-based controls are the epitome of encapsulation. Developers can manipulate control objects Programmatically using code to customize their appearance, provide data to display, and even react to events. The low-level HTML markup that these controls render is hidden away behind the scenes.

6. ASP.NET Supports all Browsers: Different browsers, versions, and configurations differ in their support of XHTML, CSS, and JavaScript. Web developers need to choose whether they should render their content according to the lowest common denominator, and whether they should add ugly hacks to deal with well-known quirks on popular browsers.

ASP.NET addresses this problem in a remarkably intelligent way. Although we can retrieve information about the client browser and its capabilities in an ASP.NET page, ASP.NET actually encourages developers to ignore these considerations and use a rich suite of web server controls.

7. ASP.NET Is Easy to Deploy and Configure: One of the biggest headaches a web developer faces during a development cycle is deploying a completed application to a production server. Not only do the web-page files, databases, and components need to be transferred, but also components need to be registered and a slew of configuration, settings need to be re-created. ASP.NET simplifies this process considerably. Every installation of the .NET Framework provides the same core classes. As a result, deploying an ASP.NET application is relatively simple. For no-frills deployment, we simply need to copy all the files to a virtual directory on a production server (using an FTP program or even a command-line command like XCOPY). As long as the host machine has the .NET Framework, there are no time-consuming registration steps.

ASP.NET File Types

File Type	Description
Ends with .aspx	These are ASP.NET web pages. They contain the user interface and, optionally, the underlying application code. Users request or navigate directly to one of these pages to start web application.

Ends with .ascx	These are ASP.NET user controls. User controls are similar to web pages, except that the user can't access these files directly. Instead, they must be hosted inside an ASP.NET web page. User controls allow us to develop a small piece of user interface and reuse it in as many web forms, as we want without repetitive code.
web.config	This is the XML-based configuration file for ASP.NET application. It includes settings for customizing security, state management, memory management, and much more.
global.asax	This is the global application file. We can use this file to define global variables (variables that can be accessed from any web page in the web application) and react to global events (such as when a web application first starts).
Ends with .cs	These are code-behind files that contain C# code. They allow us to separate the application logic from the user interface of a web page.

ASP.NET Application Directories

Every web application should have a well-planned directory structure.

Directory	Description
App_Browsers	Contains .browser files that ASP.NET uses to identify the browsers that are using our application and determine their capabilities. Usually, browser information is standardized across the entire web server, and we don't need to use this folder.
App_Code	Contains source code files that are dynamically compiled for use in our Application
App_GlobalResources	Stores global resources that are accessible to every page in the web application. This directory is used in localization scenarios, when we need to have a website in more than one language
App_LocalResources	Serves the same purpose as App_GlobalResources, except these resources are accessible to a specific page only.
App_WebReferences	Stores references to web services, which are remote code routines that a web application can call over a network or the Internet.
App_Data	Stores data, including SQL Server Express database files
App_Themes	Stores the themes that are used to standardize and reuse formatting in your web application.
Bin	Contains all the compiled .NET components (DLLs) that the ASP.NET web application uses.

ASP.Net Page Life Cycle

In general terms, the page goes through the stages outlined in the following table. In addition to the page life-cycle stages, there are application stages that occur before and after a request but are not specific to a page.

Stage

Stage	Description
Page request	The page request occurs before the page life cycle begins. When the page is requested by a user, ASP.NET determines whether the page needs to be parsed and compiled (therefore beginning the life of a page), or whether a cached version of the page can be sent in response without running the page.
Start	In the start stage, page properties such as Request and Response are set. At this stage, the page also determines whether the request is a postback or a new request and sets the IsPostBack property.

Initialization	During page initialization, controls on the page are available and each control's UniqueID property is set. A master page and themes are also applied to the page if applicable. If the current request is a postback, the postback data has not yet been loaded and control property values have not been restored to the values from view state.
Load	During load, if the current request is a postback, control property are loaded with information recovered from view state and control state.
Postback event handling	If the request is a postback, control event handlers are called. After that, the Validate method of all validator controls is called, which sets the IsValid property of individual validator controls and of the page.
Rendering	Before rendering, view state is saved for the page and all controls. During the rendering stage, the page calls the Render method for each control, providing a text writer that writes its output to the OutputStream object of the page's Response property.
Unload	The Unload event is raised after the page has been fully rendered, sent to the client, and is ready to be discarded. At this point, page properties such as Response and Request are unloaded and cleanup is performed.

INTRODUCING SERVER CONTROL

ASP.NET introduces a remarkable new model for creating web pages. In old-style web development, programmers had to master the quirks and details of HTML before they could design a dynamic web page. Pages had to be carefully tailored to a specific task, and the only way to generate additional content was to generate raw HTML tags.

ASP.NET solves this problem with a higher-level model of server controls. These controls are created and configured as objects. *They run on the web server and they automatically provide their own HTML output.* Even better, server controls behave like their Windows counterparts by *maintaining state and raising events* that we can react to in code.

ASP.NET actually provides two sets of server-side controls that we can incorporate into our web forms. These two different types of controls play subtly different roles:

1) HTML server controls: These are server-based equivalents for standard HTML elements. These controls are ideal if we're a seasoned web programmer who prefers to work with familiar HTML tags (at least at first). They are also useful when migrating ordinary HTML pages or ASP pages to ASP.NET, because they require the fewest changes. HTML controls have three main features:

i) ***They generate their own interface:*** We set properties in code, and the underlying HTML tag is created automatically when the page is rendered and sent to the client.

ii) ***They retain their state:*** Because the Web is stateless, ordinary web pages need to do a lot of work to store information between requests. HTML server controls handle this task automatically. For example, if the user selects an item in a list box, that item remains selected the next time the page is generated. Or, if our code changes the text in a button, the new text sticks the next time the page is posted back to the web server.

iii) ***They fire server-side events:*** For example, buttons fire an event when clicked, text boxes fire an event when the text they contain is modified, and so on. Our code can respond to these events, just like ordinary controls in a Windows application. With event-based programming, we can easily respond to individual user actions and create more structured code. If a given event doesn't occur, the event handler won't be executed.

2) Web controls: These are similar to the HTML server controls, but they provide a richer object model with a variety of properties for style and formatting details. They also provide more events and more closely resemble the controls used for Windows development. Web controls also feature some user interface elements that have no direct HTML equivalent, such as the GridView,

Calendar, and validation control. Following are the features of Web Controls.

i) They provide a rich user interface: A web control is programmed as an object but doesn't necessarily correspond to a single element in the final HTML page. For example, we might create a single Calendar or GridView control, which will be rendered as dozens of HTML elements in the final page. When using ASP.NET programs, we don't need to know anything about HTML. The control creates the required HTML tags for us.

ii) They provide a consistent object model: HTML is full of quirks and idiosyncrasies. For example, a simple text box can appear as one of three elements, including `<textarea>`, `<input type="text">`, and `<input type="password">`. With web controls, these three elements are consolidated as a single *TextBox* control. Depending on the properties we set, the underlying HTML element that ASP.NET renders may differ. Similarly, the names of properties don't follow the HTML attribute names. For example, controls that display text, whether it's a caption or a text box that can be edited by the user, expose a *Text* property.

iii) They tailor their output automatically: ASP.NET server controls can detect the type of browser and automatically adjust the HTML code they write to take advantage of features such as support for JavaScript. We don't need to know about the client because ASP.NET handles that layer and automatically uses the best possible set of features. This feature is known as adaptive rendering.

iv) They provide high-level features: We'll see that web controls allow us to access additional events, properties, and methods that don't correspond directly to typical HTML controls. ASP.NET implements these features by using a combination of tricks.

DATA ACCESS

The .NET Framework includes its own data access technology: ADO.NET. ADO.NET consists of managed classes that allow .NET applications to connect to data sources (usually relational databases), execute commands, and manage disconnected data. The small miracle of ADO.NET is that it allows us to write more or less the same data access code in web applications that we write for client-server desktop applications, or even single-user applications that connect to a local database.

Database Access without ADO.NET

In ASP.NET, there are a few ways to get information out of a database without directly using the ADO.NET classes. Depending on needs, we may be able to use one or more of these approaches to supplement our database code (or to avoid writing it altogether).

Options for database access without ADO.NET include the following:

- **The *SqlDataSource* control:** The *SqlDataSource* control allows us to define queries declaratively. We can connect the *SqlDataSource* to rich controls such as the *GridView*, and give our pages the ability to edit and update data without requiring any ADO.NET code. Best of all, the *SqlDataSource* uses ADO.NET behind the scenes, and so it supports any database that has a full ADO.NET provider.

However, the *SqlDataSource* is somewhat controversial, because it encourages us to place database logic in the markup portion of our page. Many developers prefer to use the *ObjectDataSource* instead, which gives similar data binding functionality but relies on a custom database component. When we use the *ObjectDataSource*, it's up to us to create the database component and write the back-end ADO.NET code.

- **LINQ to Entities:** With LINQ to Entities, we generate a data model with the design support in Visual Studio. The appropriate database logic is generated automatically. LINQ to Entities supports updates, generates secure and well-written SQL statements, and provides wide-ranging customizability. LINQ to Entities is also the preferred successor to the simpler LINQ to SQL model, which ASP.NET developers have used in the past.

ASP.NET developers will need to write some ADO.NET code, even if it's only to optimize a performance-sensitive task or to perform a specific operation that wouldn't otherwise be possible. Also, every professional ASP.NET developer needs to understand how the ADO.NET plumbing works in order to evaluate when it's required and when another approach is just as effective.

The ADO.NET Architecture

ADO.NET uses a multilayered architecture that revolves around a few key concepts, such as *Connection*, *Command*, and *DataSet* objects. One of the key differences between ADO.NET and some other database technologies is how it deals with the challenge of different data sources. In many previous database technologies, such as classic ADO, programmers use a generic set of objects no matter what the underlying data source is. For example, if we want to retrieve a record from an Oracle database using ADO code, we use the same *Connection* class we would use to tackle the task with SQL Server. This isn't the case in ADO.NET, which uses a data provider model.

ADO.NET Data Providers

A data provider is a set of ADO.NET classes that allows us to access a specific database, execute SQL commands, and retrieve data. Essentially, a *data provider is a bridge between our application and a data source*.

The classes that make up a data provider include the following:

- *Connection*: Use this object to establish a connection to a data source.
- *Command*: Use this object to execute SQL commands and stored procedures.
- *DataReader*: This object provides fast read-only, forward-only access to the data retrieved from a query.
- *DataAdapter*: This object performs two tasks. First, *we can use it to fill a DataSet* (a disconnected collection of tables and relationships) *with information extracted from a data source*. Second, *we can use it to apply changes to a data source, according to the modifications we've made in a DataSet*.

ADO.NET doesn't include generic data provider objects. Instead, it includes different data providers specifically designed for different types of data sources. Each data provider has a specific implementation of the *Connection*, *Command*, *DataReader*, and *DataAdapter* classes that's optimized for a specific RDBMS (relational database management system). For example, if we need to create a connection to a SQL Server database, we'll use a connection class named *SqlConnection*.

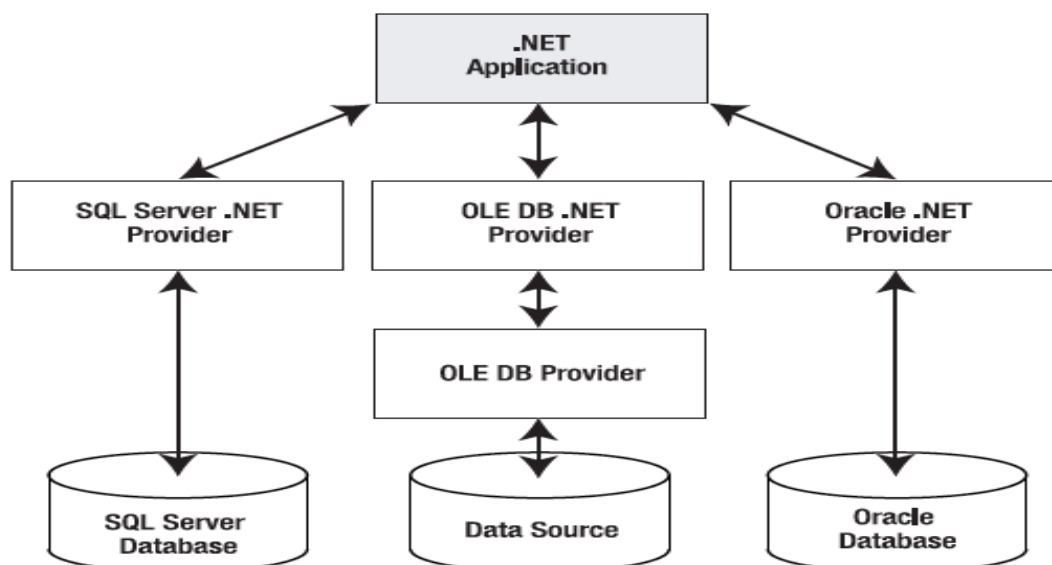


Fig: ADO.Net Architecture

Fundamental ADO.NET Classes

ADO.NET has two types of objects: connection-based and content-based.

Connection-based objects: These are the data provider objects such as *Connection*, *Command*, *DataReader*, and *DataAdapter*. They allow us to *connect to a database, execute SQL statements, move through a read-only result set, and fill a DataSet*.

Content-based objects: These objects are really just “packages” for data. They include the *DataSet*, *DataColumn*, *DataRow*, *DataRelation*, and *several others*. They are completely independent of the type of data source and are found in the System.Data namespace.

Namespace	Description
System.Data	Contains the key data container classes that model columns, relations, tables, datasets, rows, views, and constraints. In addition, contains the key interfaces that are implemented by the connection-based data objects.
System.Data.Common	Contains base, mostly abstract classes that implement some of the interfaces from System.Data and define the core ADO.NET functionality. Data providers inherit from these classes (such as DbConnection, DbCommand, and so on) to create their own specialized versions.
System.Data.OleDb	Contains the classes used to connect to an OLE DB provider, including OleDbCommand, OleDbConnection, OleDbDataReader, and OleDbDataAdapter. These classes support most OLE DB providers, but not those that require OLE DB version 2.5 interfaces.
System.Data.SqlClient	Contains the classes you use to connect to a Microsoft SQL Server database, including SqlCommand, SqlConnection, SqlDataReader, and SqlDataAdapter. These classes are optimized to use the TDS interface to SQL Server.
System.Data.OracleClient	Contains the classes required to connect to an Oracle database (version 8.1.7 or later), including OracleCommand, OracleConnection, OracleDataReader, and OracleDataAdapter. These classes are using the optimized Oracle Call Interface (OCI).
System.Data.Odbc	Contains the classes required to connect to most ODBC drivers. These classes include OdbcCommand, OdbcConnection, OdbcDataReader, and OdbcDataAdapter. ODBC drivers are included for all kinds of data sources and are configured through the Data Sources icon in the Control Panel.
System.Data.SqlTypes	Contains structures that match the native data types in SQL Server. These classes aren't required but provide an alternative to using standard .NET data types, which require automatic conversion.

Table: The ADO.Net Namespace

THE CONNECTION CLASS

The Connection class allows us to establish a connection to the data source that we want to interact with.

Connection Strings

When we create a Connection object, we need to supply a connection string. The connection string

is a series of name/value settings separated by semicolons (;). The order of these settings is unimportant, as is the capitalization.

Although connection strings vary based on the RDBMS and provider we are using, a few pieces of information are almost always required:

The server where the database is located: In the examples, the database server is always located on the same computer as the ASP.NET application, so the loopback alias localhost is used instead of a computer name.

The database we want to use: Most of the examples we are use the Northwind database, which is installed with older versions of SQL Server

How the database should authenticate: The Oracle and SQL Server providers give us the choice of supplying authentication credentials or logging in as the current user. The latter choice is usually best, because we don't need to place password information in our code or configuration files.

For example, here's the connection string we would use to connect to the Northwind database on the current computer using integrated security (which uses the currently logged-in Windows user to access the database):

```
string connectionString = "Data Source=localhost; Initial Catalog=Northwind;" +  
"Integrated Security=SSPI";
```

If integrated security isn't supported, the connection must indicate a valid user and password combination.

```
string connectionString = "Data Source=localhost; Initial Catalog=Northwind;" + "user id=sa;  
password=opensesame";
```

The <connectionStrings> section of the web.config file is a handy place to store the connection strings.

Here's an example:

```
<configuration>  
<connectionStrings>  
<add name="DBConnectionString" connectionString=  
"Data Source=localhost; Initial Catalog=Northwind; Integrated Security=SSPI"/>  
</connectionStrings>
```

...

```
</configuration>
```

We can then retrieve our connection string by name from the *WebConfigurationManager.ConnectionStrings* collection

Testing a Connection

Once we've chosen our connection string, managing the connection is easy—we simply use the *Open()* and *Close()* methods.

We can use the following code in the *Page.Load* event handler to test a connection and write its status to a label. To use this code as written, we must import the *System.Data.SqlClient* namespace.

// Create the Connection object.

```
string connectionString =  
WebConfigurationManager.ConnectionStrings["DBConnectionString"].ConnectionString;  
SqlConnection con = new SqlConnection(connectionString);  
try  
{  
    // Try to open the connection.  
    con.Open();  
    lblInfo.Text = "<b>Server Version:</b> " + con.ServerVersion;  
    lblInfo.Text += "<br /><b>Connection Is:</b> " + con.State.ToString();  
}
```

```
catch (Exception err)
{
    // Handle an error by displaying the information.
    lblInfo.Text = "Error reading the database. " + err.Message;
}
finally
{
    // Either way, make sure the connection is properly closed.
    // Even if the connection wasn't opened successfully,
    // calling Close() won't cause an error.
    con.Close();
    lblInfo.Text += "<br /><b>Now Connection Is:</b> " +
    con.State.ToString();
}
```

THE COMMAND AND DATAREADER CLASSES

The Command class allows us to execute any type of SQL statement. Although we can use a Command class to perform data definition tasks (such as creating and altering databases, tables, and indexes), we're much more likely to perform data manipulation tasks (such as retrieving and updating the records in a table).

Command Basics

Before we can use a command, we need to choose the *command type*, set the *command text*, and *bind the command to a connection*. We can perform this work by setting the corresponding properties (CommandType, CommandText, and Connection), or we can pass the information we need as constructor arguments.

The command text can be a *SQL statement*, a *stored procedure*, or the *name of a table*. It all depends on the type of command we're using. Three types of commands exist.

Value	Description
CommandType.Text	The command will execute a direct SQL statement. The SQL statement is provided in the CommandText property. This is the default value.
CommandType.StoredProcedure	The command will execute a stored procedure in the data source. The CommandText property provides the name of the stored procedure.
CommandType.TableDirect	The command will query all the records in the table. The CommandText is the name of the table from which the command will retrieve the records. (This option is included for backward compatibility with certain OLE DB drivers only. It is not supported by the SQL Server data provider, and it won't perform as well as a carefully targeted query.)

Table: Values for the CommandType Enumeration

For example, here's how we would create a Command object that represents a query:

```
SqlCommand cmd = new SqlCommand();
cmd.Connection = con;
cmd.CommandType = CommandType.Text;
cmd.CommandText = "SELECT * FROM Employees";
```

And here's a more efficient way using one of the Command constructors. Note that we don't need to specify the CommandType, because CommandType.Text is the default.

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Employees", con);
```

Alternatively, to use a stored procedure,

```
SqlCommand cmd = new SqlCommand("GetEmployees", con);
```

```
cmd.CommandType = CommandType.StoredProcedure;
```

These examples simply define a Command object; they don't actually execute it. The Command object provides three methods that we can use to perform the command, depending on whether we want to retrieve a full result set, retrieve a single value, or just execute a nonquery command.

Method	Description
ExecuteNonQuery()	Executes non-SELECT commands, such as SQL commands that insert, delete, or update records. The returned value indicates the number of rows affected by the command. You can also use ExecuteNonQuery() to execute data-definition commands that create, alter, or delete database objects (such as tables, indexes, constraints, and so on).
ExecuteScalar()	Executes a SELECT query and returns the value of the first field of the first row from the rowset generated by the command. This method is usually used when executing an aggregate SELECT command that uses functions such as COUNT() or SUM() to calculate a single value.
ExecuteReader()	Executes a SELECT query and returns a DataReader object that wraps a read-only, forward-only cursor.

Table: Command Methods

The DataReader Class

A *DataReader* allows us to read the data returned by a SELECT command one record at a time, in a forward-only, read-only stream. This is sometimes called a firehose cursor. Using a DataReader is the simplest way to get to our data, but it lacks the sorting and relational abilities of the disconnected DataSet.

Methods	Description
Read()	Advances the row cursor to the next row in the stream. This method must also be called before reading the first row of data. The Read() method returns true if there's another row to be read, or false if it's on the last row.
GetValue()	Returns the value stored in the field with the specified index, within the currently selected row.
GetValues()	Saves the values of the current row into an array. The number of fields that are saved depends on the size of the array we pass to this method.
GetInt32(), GetChar(), GetDateTime(), GetXxx()	These methods return the value of the field with the specified index in the current row, with the data type specified in the method name.
NextResult()	If the command that generated the DataReader returned more than one rowset, this method moves the pointer to the next rowset.
Close()	Closes the reader.

Example: Filling the ListBox

To start, the connection string is defined as a private variable for the page class and retrieved from the connection string:

```
private string connectionString =
```

```
WebConfigurationManager.ConnectionStrings["DBConnectionString"].ConnectionString;
```

The list box is filled when the Page.Load event occurs. Because the list box is set to persist its view state information, this information needs to be retrieved only once—the first time the page is displayed.

It will be ignored on all postbacks. Here's the code that fills the list from the database:

```
protected void Page_Load(Object sender, EventArgs e)
```

```
{
if (!this.IsPostBack)
{
    FillAuthorList();
}
}
private void FillAuthorList()
{
    lstAuthor.Items.Clear();
    // Define the Select statement.
    // Three pieces of information are needed: the unique id
    // and the first and last name.
    string selectSQL = "SELECT au_lname, au_fname, au_id FROM Authors";
    // Define the ADO.NET objects.
    SqlConnection con = new SqlConnection(connectionString);
    SqlCommand cmd = new SqlCommand(selectSQL, con);
    SqlDataReader reader;
    // Try to open database and read information.
try
{
    con.Open();
    reader = cmd.ExecuteReader();
    // For each item, add the author name to the displayed
    // list box text, and store the unique ID in the Value property.
    while (reader.Read())
    {
        ListItem newItem = new ListItem();
        newItem.Text = reader["au_lname"] + ", " + reader["au_fname"];
        newItem.Value = reader["au_id"].ToString();
        lstAuthor.Items.Add(newItem);
    }
    reader.Close();
}
catch (Exception err)
{
    lblResults.Text = "Error reading list of names. ";
    lblResults.Text += err.Message;
}
finally
{
    con.Close();
}
}
```